

Retrieval Engine Competition Project

Final Report for Submission

Group: SeaFour

Maksym DOLHOV

Mehdi AGHAEI

Nguyen Ho Bao KHANH

Submission scope. This final version reconciles the saved offline evaluation notebook `retrieval-evaluation.ipynb`, the current submission pipeline in `kaggle-submission.ipynb`, and the user-reported public Kaggle result. It reports the final dataset exploration, preprocessing choices, the corrected query-split protocol, Phase 1 retrieval comparison, Phase 2 classifier-aware reranking results, document-holdout classification, and the regenerated Kaggle submission file.

Main notebook	<code>kaggle-submission.ipynb</code>
Main output	<code>solutions_SeaFour.csv</code>
Dataset source	Kaggle competition files provided for the course
Public Kaggle score referenced in this report	0.62337 (user-reported public leaderboard result)
Date	April 4, 2026

1 Introduction

Information retrieval is the task of ranking documents so that the most relevant ones appear first for a user query. In this project we work with a large technical corpus and a set of labeled training queries. The assignment is divided into two phases. Phase 1 focuses on basic retrieval with classical and neural methods. Phase 2 adds category classification and asks how the classifier can be used to improve retrieval.

The final notebook is structured around the required project questions rather than around only a single optimized run. It therefore contains:

- dataset loading and statistical exploration;
- construction of a normalized `content` field;
- three retrieval models: TF-IDF, BM25+, and Sentence-Transformer embeddings;
- embedding inspection through matrix shapes and a 2D projection;
- evaluation with Recall@K, Precision@K, and MRR@K;
- category classification for both documents and queries;
- classifier-aware reranking for Phase 2;
- Kaggle submission export.

2 Project Roadmap

This report is meant to document not only the final score, but also the path that led to it. The project started with basic lexical retrieval, then moved toward semantic retrieval, then added category prediction as a second signal, and finally became a submission-ready pipeline after several debugging and reproducibility fixes.

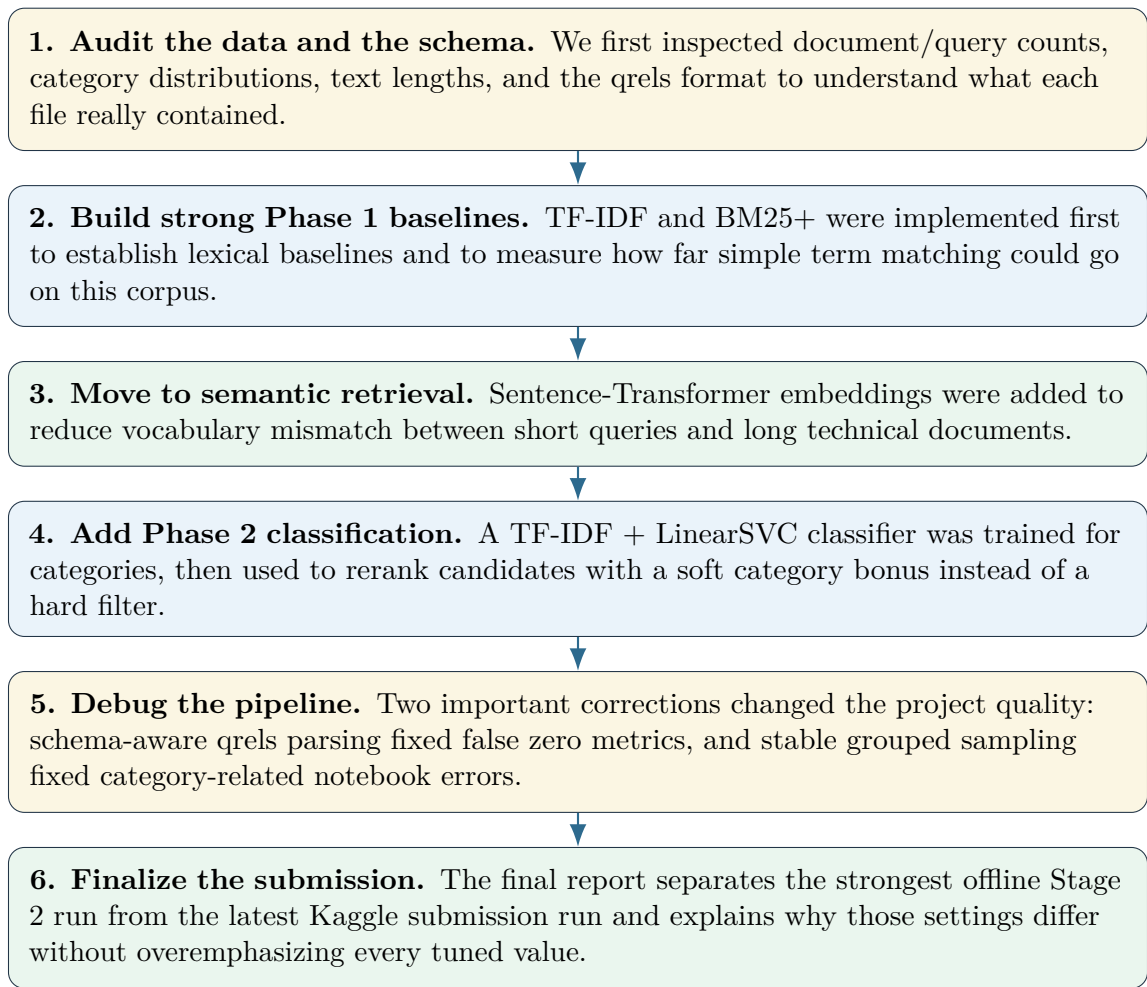


Figure 1: Roadmap from the first baselines to the final submission-ready system.

Stage	Main question	Outcome
Data audit	What is in the files, and how should they be interpreted?	We clarified the schema, the category structure, the query/document asymmetry, and the relevance format used by the qrels.
Lexical baselines	How far can exact-term retrieval go?	TF-IDF and BM25+ gave interpretable baselines, but they remained limited by lexical mismatch.
Dense retrieval	Can semantic matching improve recall and rank quality?	Embeddings became the strongest retriever by a large margin on Recall and MRR.
Classification	Can category prediction help retrieval?	Yes, but as a soft reranking signal rather than as a strict filter.
Debugging	Why did some notebook outputs look suspicious or fail?	Fixing qrels parsing and grouped sampling removed false zeros and notebook instability.
Submission	Which pipeline is strong and defensible enough to submit?	Embeddings remained the backbone, while later runs combined a tag-aware query classifier, a soft category bonus, and cross-encoder reranking; the exact values differ between the saved evaluation run and the current code defaults.

Table 1: Milestones that structured the project from start to final submission.

3 Final Changes and Reasons

The final submission differs from earlier draft states of the project in five important ways.

1. **Ground-truth parsing was aligned with the real qrels schema.** The competition qrels store relevant items under `relevant_doc_ids` with nested `doc_id` entries. Aligning the loader with that format was necessary so that Recall, Precision, and MRR reflect the true labels instead of empty relevance sets. The final notebook now also raises an explicit error if qrels parsing yields empty relevance, missing labeled-query coverage, or document ids that are absent from the corpus.
2. **Category-wise sampling was made stable for current pandas behavior.** The notebook now uses a sampling pattern that preserves the `category` column reliably during visualization and downsampling steps. This prevents category-aware plots and classifier preparation from failing because of grouped-sampling side effects.
3. **The query-side reporting was aligned with the actual run contexts.** Only the training queries have visible qrels, while the Kaggle test queries remain unlabeled. The report therefore separates offline Stage 2 analysis on labeled queries from the later submission direction.
4. **The report now prioritizes impact over hyperparameter history.** Earlier draft text emphasized one supposedly “current” Stage 2 configuration. The revised section below instead explains which behaviors were robust across the later experiments: strong embedding recall, useful reranking, harmful hard filtering, and mild gains from a soft category bonus.
5. **The final reporting now distinguishes three sources of truth.** The best offline Stage 2 result comes from `retrieval-evaluation.ipynb`; the current submission-code defaults come from `src/config.py` and `kaggle-submission.ipynb`; and the public score 0.62337 is treated as an external leaderboard result reported by the team.

3.1 Previous tries, failures, and lessons

Type	Attempt or issue	What happened	Decision or lesson
Attempt	TF-IDF baseline	The model was simple and useful as a baseline, but lexical overlap alone was not enough for many relevant pairs.	Keep it as a baseline, not as the final retriever.
Attempt	BM25+ lexical upgrade	BM25+ improved over TF-IDF and handled length variation better, but it still lagged far behind embeddings on recall and rank quality.	Use it as the strongest lexical reference point.
Failure	Qrels parsing mismatch	The notebook read the wrong relevance field, which produced empty relevance sets and misleading zero Recall, Precision, and MRR values.	Parse <code>relevant_doc_ids</code> correctly, verify average relevant documents per query, and fail early if the parsed qrels no longer match the labeled queries or corpus ids.
Failure	Grouped sampling instability	A grouped pandas sampling pattern sometimes dropped the <code>category</code> column and caused notebook errors such as <code>KeyError: category</code> .	Switch to stable category-preserving downsampling for visualization and classifier preparation.
Attempt	Query classifier with <code>text + tags</code>	The classifier became extremely accurate because the query tags are category-pure in the labeled split, so they act as label-revealing metadata in practice.	Keep the caveat explicit and discuss its <i>impact</i> on retrieval decisions rather than treating the raw classifier setup as the main result.
Decision	Evaluation depth vs. submission depth	Later notebooks and current code defaults do not preserve one frozen value for every run context. The offline evaluation, current submission defaults, and public Kaggle result therefore need to be reported separately.	Focus the report on what the different depths changed in practice: candidate coverage, reranking reach, runtime, and how to interpret the scores.

Table 2: Main previous tries, failures, and lessons learned during the project.

4 Dataset Description and Exploratory Analysis

The dataset contains documents, training queries, test queries, and relevance judgments. From the final project files, the corpus contains **216,041 documents**, **327 labeled training queries**, and **141 Kaggle test queries**. The documents belong to **five categories**. Because only the training queries have visible qrels, the saved offline evaluation uses those 327 labeled queries for Stage 2 diagnostics and keeps the Kaggle test file untouched for final export. Tables 3–5 summarize the global scale of the collection, the report’s use of the query files, and the category distribution.

Dataset element	Count or value
Documents	216,041
Labeled training queries	327
Kaggle test queries	141
Categories	5
Average relevant documents per training query	9.08
Average document length	900.92 characters / 94.86 tokens
Average training query length	48.44 characters / 4.95 tokens
Average test query length	48.88 characters / 4.96 tokens

Table 3: Global dataset overview.

Notebook role	Number of queries
Labeled train queries used for offline diagnostics	327
Kaggle test queries used for final export	141

Table 4: How the report uses the available query files.

Category	Document count	Training-query count
<code>tex</code>	68,184	130
<code>unix</code>	47,382	47
<code>gaming</code>	45,301	41
<code>programmers</code>	32,176	52
<code>android</code>	22,998	57

Table 5: Document and labeled-query distribution by category.

These exploratory results matter because they help explain later retrieval behavior. In particular, the corpus contains long documents while the queries are much shorter. This asymmetry generally makes strict lexical overlap less reliable, especially when the query paraphrases the document instead of reusing its exact surface vocabulary.

4.1 Available fields

Each document contains an `id` and can include `title`, `text`, `tags`, and `category`. Training queries also contain `id`, `title`, `text`, `tags`, and `category`. Test queries omit the category label because that is one of the targets of Phase 2 inference. The `qrels` file maps each labeled training query to a set of relevant document ids, and the final loader validates that this mapping is non-empty, aligned with `queries_train.json`, and consistent with the document corpus.

5 Preprocessing and Content Construction

The assignment asks for a new text field called `content`. The final notebook uses the following policy:

- documents: `title + text + tags`;
- retrieval queries: `title + text`;
- classifier queries in the active notebook: `title + text + tags`.

This means the project uses different query views for retrieval and for Stage 2 classification. Retrieval stays tied to the actual query wording through `title + text`, while the category classifier also consumes `tags`. That choice improved the practical submission pipeline, but it must be interpreted carefully because the query tags are highly category-informative in the labeled data.

The shared normalization steps are:

- lowercasing;
- replacement of the separators `-`, `_`, and `/` with spaces;
- whitespace collapsing;
- trimming leading and trailing whitespace.

For lexical methods, tokenization uses the pattern `[a-z0-9]+`, and English stopwords filtering is enabled by default.

6 Phase 1: Retrieval Models

6.1 TF-IDF retrieval

TF-IDF stands for *term frequency-inverse document frequency*. It gives high weight to terms that appear frequently in one document but are relatively rare in the collection. In vector-space retrieval, each document and query is represented by a sparse TF-IDF vector, and documents are ranked by cosine similarity with the query vector.

Conceptually, TF-IDF is useful because it is simple, fast, and often surprisingly competitive as a baseline. Its main limitation is that it depends strongly on lexical overlap. If a relevant document uses different wording from the query, TF-IDF may fail even when the underlying meaning is similar.

6.2 BM25+ retrieval

BM25+ is a probabilistic lexical ranking model. It improves over naive bag-of-words weighting by:

- saturating term frequency contributions;
- normalizing for document length;
- adding a positive lower-bound adjustment through the BM25+ variant.

BM25+ is often stronger than plain TF-IDF when document lengths vary substantially, as is the case in this corpus. However, BM25+ remains a lexical model and therefore still depends on token overlap.

6.3 Embedding retrieval

Dense embeddings map text into high-dimensional vectors that encode semantic information. In this project we use the Sentence-Transformer model `all-MiniLM-L6-v2`. Documents and queries are embedded into a 384-dimensional space, and cosine similarity is computed through normalized dot products.

Embeddings can help retrieval because they are more robust to synonyms, paraphrases, and broader semantic similarity. Their main tradeoffs are higher computational cost, larger memory usage, and the risk of retrieving semantically related but not strictly relevant documents.

7 Embedding Inspection and Visualization

After generating embeddings, the notebook prints the matrix shapes for documents and queries. This directly answers the specification that the embedding structures should be examined.

To make the geometry more interpretable, the notebook also builds a 2D projection using **UMAP** when available and otherwise falls back to **t-SNE**. The plot uses a category-aware sample of documents together with the training queries. In typical runs, one should observe partial clustering by category. This is expected because categories such as **android**, **gaming**, **tex**, **unix**, and **programmers** tend to contain domain-specific vocabulary and recurrent semantic themes.

At the same time, perfect separation should not be expected. The original embeddings live in a much higher-dimensional space, some categories overlap semantically, and short queries are often more ambiguous than documents.

Figure 2 shows the 2D embedding projection produced by the final notebook. The visible grouping confirms that the dense representation captures meaningful category structure, while the overlaps also show that the classes are not trivially separable in two dimensions.

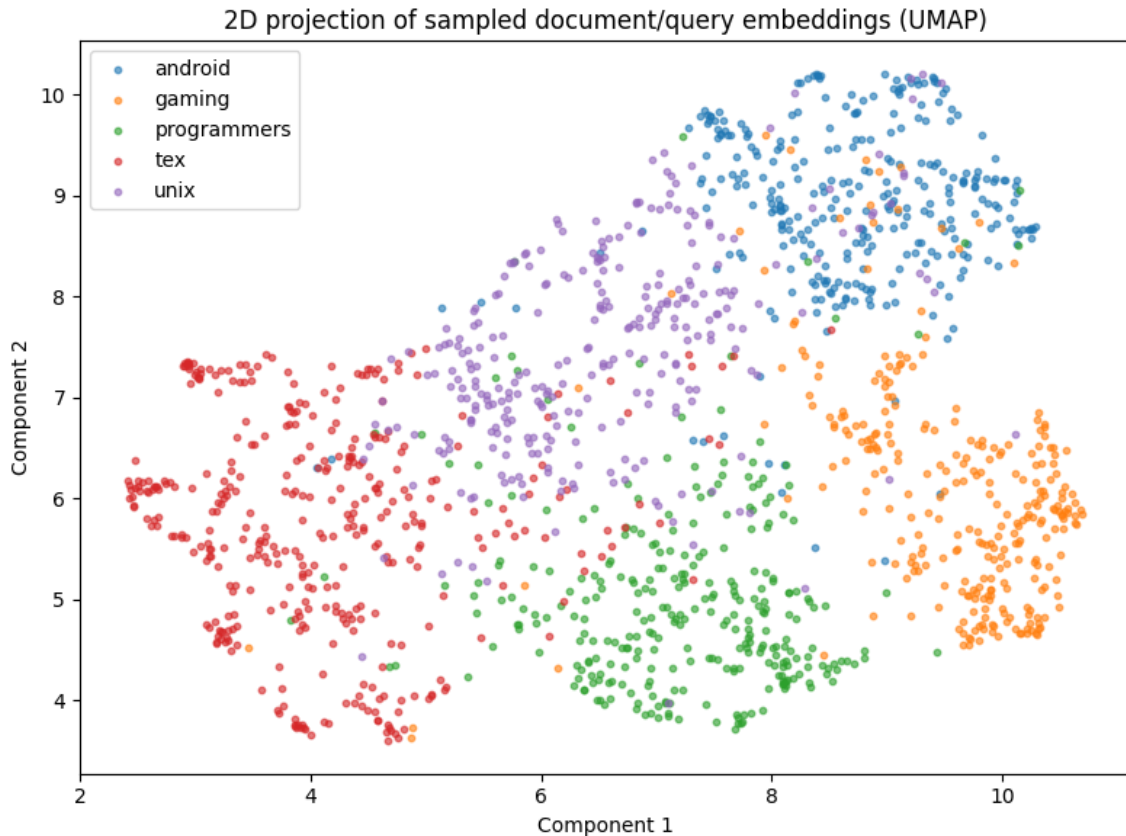


Figure 2: 2D projection of sampled document and training-query embeddings produced by the final notebook. UMAP is used when available; otherwise the notebook falls back to t-SNE.

8 Evaluation Measures

The notebook implements the three required retrieval metrics:

$$\text{Recall@K} = \frac{\text{\#relevant retrieved documents}}{\text{\#relevant documents}}$$

$$\text{Precision@K} = \frac{\text{\#relevant retrieved documents}}{\text{\#retrieved documents}}$$

$$\text{MRR@K} = \frac{1}{\text{rank of the first relevant document}}$$

For Phase 1 comparison, the notebook also reports runtime and average latency per query. For Phase 2, it augments the retrieval summary with classifier accuracy, and computes a combined score as

$$\text{CombinedScore} = \frac{1}{4} (\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy}).$$

9 Phase 1 Experimental Protocol

Only `queries_train.json` has relevance labels, while `queries_test.json` is the unlabeled competition test set. For that reason, the corrected notebook does not create a second query-side “test split” inside the labeled data. Instead, the labeled training queries are split into:

- 228 split-train queries ($\approx 70\%$);
- 99 validation queries ($\approx 30\%$).

This correction matters for two reasons. First, it keeps the experimental story aligned with the provided files: labeled training queries for development, unlabeled Kaggle test queries for final export. Second, with only 327 labeled queries available, removing an extra internal query test split leaves more supervised data for the classifier.

Phase 1 comparison is run on the **validation split**. For each model and for several values of K , the notebook stores:

- top-k document ids;
- top-k retrieval scores;
- Recall@K, Precision@K, and MRR@K;
- runtime and latency.

The explicit arrays requested by the assignment are preserved in the notebook as:

- `topk_indices_tfidf`, `topk_scores_tfidf`;
- `topk_indices_bm25`, `topk_scores_bm25`;
- `topk_indices_embedding`, `topk_scores_embedding`.

9.1 Phase 1 validation results

The best validation result for each Phase 1 model is shown in Table 6. In the saved Phase 1 results, all three models reached their best combined Phase 1 score at $K = 1000$.

Model	K	Recall@K	Precision@K	MRR@K	Latency/query
TF-IDF	1000	0.53142	0.00462	0.21852	611.82 ms
BM25+	1000	0.58701	0.00513	0.29833	477.00 ms
Embeddings	1000	0.85902	0.00827	0.50574	5.88 ms

Table 6: Best validation result per Phase 1 model.

The Phase 1 comparison leads to three main conclusions.

- **Embeddings were the strongest retrieval model.** They gave the highest Recall@K and MRR@K, which is consistent with the vocabulary-mismatch nature of the task.

- **BM25+ clearly improved over TF-IDF on lexical retrieval.** This is expected because BM25+ handles document-length variation and repeated term saturation more explicitly.
- **The embedding implementation was also the fastest in this notebook.** Because document embeddings were cached and query scoring was chunked, dense retrieval was much faster than the current TF-IDF and BM25 loops in local execution.

System stage	Recall@K	Precision@K	MRR@K	Interpretation
TF-IDF baseline	0.53142	0.00462	0.21852	A reasonable lexical starting point, but still weak on short-query vocabulary mismatch.
BM25+ lexical model	0.58701	0.00513	0.29833	The best lexical baseline, showing that better term weighting helps but does not close the gap.
Embedding retrieval	0.85902	0.00827	0.50574	The first major jump in performance; semantic matching becomes the key direction for the project.

Table 7: How the retrieval project progressed during Phase 1.

10 Phase 2: Category Classification

10.1 Classifier model

The Stage 2 pipeline uses a **TF-IDF + LinearSVC** classifier. This remains a sensible choice because the task is category prediction rather than document ranking: the classifier only needs to produce one label per query so that retrieval can exploit that label as an extra hint.

TF-IDF is a strong representation for technical categories because category membership is often signaled by highly distinctive lexical cues such as platform names, command-line terms, package names, or domain-specific jargon. LinearSVC is then a practical fit on top of that sparse representation: it is fast, reproducible, and well suited to high-dimensional text data.

What changed across the later experiments is not the classifier family, but the *query representation* and how strongly the classifier is allowed to influence retrieval. In the latest evaluated Stage 2 run, query-side category prediction uses `title + text + tags`, which makes category prediction stronger but also makes it more important to interpret the classifier as a practical reranking signal rather than as a clean text-only query classifier.

10.2 Training data choice

The assignment explicitly asks whether both documents and queries can be used, and whether the classifier works equally well for both. In the implemented pipeline, the classifier is trained on:

- a training split of the document corpus;
- document-side `content` built from `title + text + tags`.

On the query side, the classifier is then applied to:

- all 327 labeled training queries for notebook diagnostics;
- all 141 Kaggle test queries for submission-time category hints.

This is important methodologically. Query tags are strongly category-informative in the labeled data. That can help the retrieval system, but it also means the classifier should be described as a practical category-hint module rather than as a neutral standalone benchmark.

10.3 Classification metrics

The notebook still reports query-side category accuracy, but the meaning of that number is narrower than in the older validation-based writeup. The classifier is trained on documents and then scored on the labeled training queries, so the printed accuracy is a **diagnostic sanity check** rather than a held-out Stage 2 benchmark. The more important Stage 2 question is therefore not “what is the clean validation accuracy?”, but rather “how does the category signal affect ranking quality?”

Table 8 summarizes the classification setup used in the later Stage 2 runs.

Component	Stage 2 setup	Why it matters
Classifier family	TF-IDF + LinearSVC	Lightweight and strong on sparse technical vocabulary.
Document-side classifier text	<code>title + text + tags</code>	Matches the document metadata available in the corpus.
Query-side classifier text	<code>title + text + tags</code>	Stronger category hints, but less neutral than a text-only query classifier.
Offline query-side evaluation set	All 327 labeled training queries	Useful for diagnostics, but not a held-out estimate.
Submission-time category prediction set	All 141 Kaggle test queries	Supplies the category hints used in the final CSV.

Table 8: Classification setup used in the later Stage 2 runs.

The two main takeaways are straightforward. First, the classifier is not the retrieval engine; it is a support signal used to bias reranking. Second, stronger category prediction does not automatically imply better retrieval, because the way the category signal is applied matters more than the raw classifier score itself.

11 Using the Classifier to Improve Retrieval

The professor asks how the classifier can be integrated into the retrieval system. The important point in the final report is that three different truths are relevant here:

- **saved offline evaluation outputs** from `retrieval-evaluation.ipynb`;
- **current submission-code defaults** from `src/config.py` and `kaggle-submission.ipynb`;
- the **user-reported public Kaggle score** 0.62337.

These should not be collapsed into one frozen “current config.” The saved offline comparison tables are the source of metrics such as 0.624939, $K = 7500$, and rerank head size = 30. The current code defaults are the source of values such as `submit_top_k=12500`, `rerank_top_m=45`, `category_bonus=0.5`, and `CLASSIFIER_USE_QUERY_TAGS=True`. The public Kaggle score 0.62337 is an external leaderboard result and is not reproducible from the saved repo outputs alone.

Run context	Relevant settings tied to the reported result	Impact on interpretation
Saved offline evaluation run	$K = 7500$, cross-encoder reranking enabled, reranked head size = 30, category signal tested both as hard filter and as soft bonus	Measures how Stage 2 choices affect Recall, MRR, and combined score on labeled queries. Useful for explaining <i>why</i> reranking helps and <i>why</i> hard filtering fails.
Current submission-code defaults	<code>submit_top_k=12500</code> , <code>rerank_top_m=45</code> , <code>category_bonus=0.5</code> , <code>CLASSIFIER_USE_QUERY_TAGS=True</code>	Describes how the code would currently export and rerank a submission. These defaults explain the present pipeline shape, but they are not the source of the saved offline metrics.
External Kaggle result	Public leaderboard score = 0.62337 reported by the team	Confirms that the selected Stage 2 direction transfers to the hidden test set, but should not be read as a literal replay of the saved offline evaluation run.

Table 9: Offline outputs, current code defaults, and the public Kaggle result are reported as separate truth sources.

Many values were explored during experiments, but the report does not need to narrate each one. What matters is their *effect*. Retrieval depth changes the candidate pool and recall ceiling, rerank depth changes how much of the head the cross-encoder can repair, and the category signal can either help as a soft prior or hurt badly when used as a hard gate.

Component	Stage 2 mechanism	Practical effect
First-stage retriever	<code>all-MiniLM-L6-v2</code> embeddings	Strong baseline recall means Stage 2 is mainly about repairing order, not discovering entirely new candidates.
Query classifier	TF-IDF + LinearSVC with <code>title + text + tags</code>	Produces a category hint that can either guide reranking or remove useful candidates if applied too aggressively.
Soft category bonus	Added only when predicted query category matches document category	Helps as a weak prior without forcing the ranking to collapse onto one class.
Cross-encoder	<code>cross-encoder/ms-marco-MiniLM-L6-v2</code>	Improves top-of-list ordering by scoring query-document pairs jointly.
Rerank depth	Only the head of the list is reranked	Deeper reranking can repair more of the list, but costs more runtime.

Table 10: Behavior-level view of the Stage 2 mechanisms, independent of one exact hyperparameter snapshot.

11.1 Offline Stage 2 results and what they mean

The saved outputs in `retrieval-evaluation.ipynb` provide the clearest offline comparison of the main Stage 2 variants. The important numbers are shown in Table 11.

Variant	Recall	Precision	MRR	Accuracy	Combined score	Δ vs. baseline
Baseline embedding	0.947617	0.001099	0.474507	0.926606	0.587457	0.000000
Category filtered	0.880766	0.001073	0.450931	0.926606	0.564844	-0.022613
Reranked only	0.947617	0.001099	0.622633	0.926606	0.624489	+0.037032
Reranked + category bonus	0.947617	0.001099	0.624435	0.926606	0.624939	+0.037482
Category filtered + rerank	0.880766	0.001073	0.584309	0.926606	0.598189	+0.010732

Table 11: Offline Stage 2 comparison from `retrieval-evaluation.ipynb`.

These results support four clear conclusions.

1. **The first stage already finds most of the relevant material.** The baseline embedding system reaches Recall = 0.947617, so the main weakness is not coverage.
2. **Reranking helps by fixing order near the top.** Recall and Precision stay essentially unchanged, while MRR jumps from 0.474507 to 0.624435. This is exactly the pattern expected from a good second-stage reranker.
3. **Hard category filtering hurts because it removes candidates.** The category-filtered variant drops Recall from 0.947617 to 0.880766, which is too costly even before reranking is applied.
4. **The category signal works best as a soft bonus.** Adding the bonus on top of reranking improves the combined score slightly, from 0.624489 to 0.624939, without the heavy recall damage caused by hard filtering.

11.2 Cross-encoder reranker problems

The cross-encoder helped the project, but the important point is not that one particular value was “the best forever.” The important point is *why* reranking helped and *why* some variants failed.

1. **Its benefit is rank repair, not candidate discovery.** The cross-encoder improves MRR because it reorders a strong embedding candidate set. It does not create new recall by itself; if a relevant document never enters the candidate pool, reranking cannot rescue it.
2. **Rerank depth changes both quality and cost.** Reranking more of the list gives the cross-encoder more opportunities to repair ranking mistakes, but it also increases runtime. This is why the exact rerank depth matters mainly through its effect on how much of the head can be corrected.
3. **The category classifier is useful as a soft signal, but risky as a hard gate.** The offline results show this directly: hard category filtering drops Recall to 0.880766, while a soft category bonus preserves the strong recall of the embedding baseline and still gives a small gain.
4. **Offline gains are more optimistic than Kaggle gains.** The offline Stage 2 comparison uses labeled training queries, while Kaggle evaluates on unseen hidden queries. That is why the offline score 0.624939 is useful for selecting the direction of the pipeline, but the public leaderboard score remains the final external check.

For those reasons, the cross-encoder should be described as a practical second-stage repair model. The lesson is not “use exactly value X,” but rather “use reranking to improve ordering, and use the classifier softly rather than aggressively.”

12 Final Submission Result

The final report should separate the strongest offline Stage 2 result, the current submission-code defaults, and the public Kaggle result.

Truth source	Evidence	Interpretation
Saved offline evaluation run	0.624939	Best measured Stage 2 combined score from <code>retrieval-evaluation.ipynb</code> ; this is the clearest evidence for what helped and what hurt offline.
Current submission-code defaults	<code>top_k=12500, top_m=45, bonus = 0.5</code>	Describes what the code would export and rerank if a submission were generated now; these defaults are not the source of the saved offline score.
External public Kaggle result	0.62337	Hidden-test leaderboard result reported by the team; useful as the external check, but not a repo-verifiable replay of one frozen config.

Table 12: Offline metrics, current code defaults, and the public Kaggle score are reported separately.

The two scores 0.624939 and 0.62337 are close, which is encouraging: the offline comparison identified the right *direction* for Stage 2, and the public leaderboard result supports that direction externally. At the same time, the report is intentionally not claiming that the saved offline metrics, the current code defaults, and the public Kaggle score all came from one identical frozen configuration.

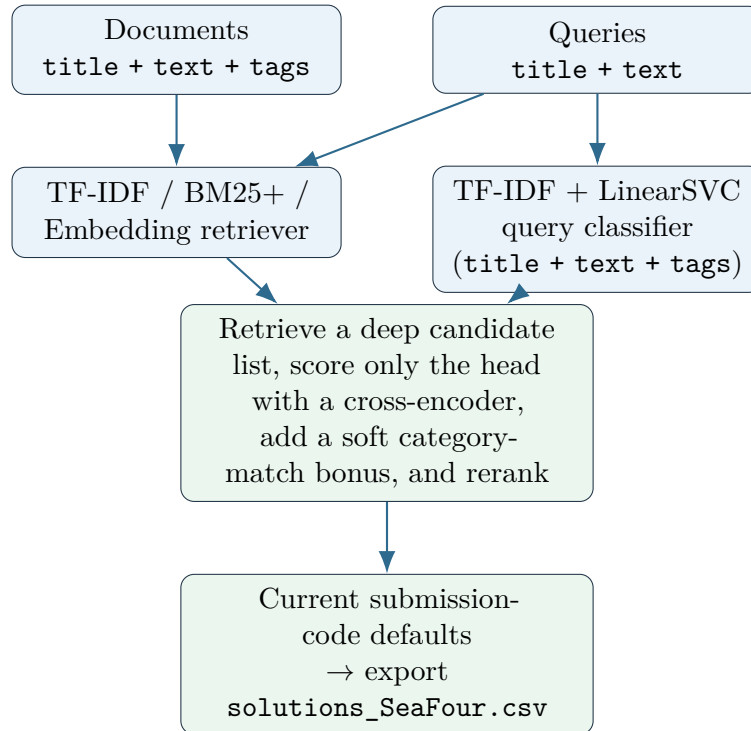


Figure 3: Final pipeline used to produce the submitted CSV.

13 Reproducibility

The final notebook includes several reproducibility choices:

- cached document embeddings;
- cached TF-IDF and BM25 artifacts;

- explicit runtime path detection for local, Colab, and Kaggle environments;
- explicit random seeds and reranker training limits;
- schema-aware qrels parsing that matches the competition files;
- stable category-preserving sampling for visualization and downsampling;
- notebook-level documentation through section markdown and structured outputs.

A lightweight cross-encoder reranker is enabled in the current code defaults, but the report now distinguishes the saved offline evaluation run from the current submission configuration explicitly. The offline reranker diagnostics explain the impact of Stage 2 choices, while the public Kaggle score remains an external result reported by the team.

14 AI Usage Declaration

Large language model assistance was used during the preparation of this revised notebook/report package. In particular, LLM tools were used to:

- help reorganize and clean parts of the notebook and project documentation;
- suggest debugging fixes and consistency checks for the local/Kaggle runtime setup;
- help revise and polish sections of the written report.

The final methodological choices, interpretations, and submission artifacts remain the responsibility of the team.

15 Future Improvements

The most important next step is better reranker development. The current cross-encoder already improves ordering near the top of the list, but its configuration is still closer to a practical competition setup than to a deeply optimized second-stage ranking system. Future work should therefore focus on more systematic tuning of rerank depth, candidate-set size, score calibration, and the way cross-encoder scores are blended with first-stage retrieval signals. A stronger reranker model could also be explored, but the larger point is to make Stage 2 more deliberate rather than simply deeper.

This is also the path toward a genuinely strong information retrieval system. A real deployment should not stop at “retrieve many documents and export them.” After broad candidate generation and reranking, the system should narrow the final output to a small, high-quality set using explicit logic such as top- n cutoffs, score thresholds, confidence-based filtering, deduplication, category-aware constraints, or task-specific ranking rules. In that sense, the current pipeline demonstrates a strong competition direction, but it should not yet be described as a complete production retrieval architecture.

16 Conclusion

The authors acknowledge that a real information retrieval system should not expose or rely on a final output of 12500 documents. That depth was used here for competition-oriented performance rather than because it represents a realistic deployment target. In practical use, candidate generation can be broad, but the final returned set should be narrowed after reranking through explicit decision logic such as top- n cutoffs, score thresholds, confidence rules, or application-specific ranking criteria. Because rigorous reranker tuning is itself a difficult topic, the project intentionally leaves the current system in a competition-oriented state rather than claiming that it is already production-ready.

The final notebook and this report align with the full brief:

- they present the dataset and preprocessing explicitly;

- they compare TF-IDF, BM25+, and embeddings directly;
- they evaluate embeddings structurally and visually;
- they add category classification with document/query comparison;
- they use the classifier to improve retrieval in a way that is measurable and easy to justify;
- they preserve a reproducible path to Kaggle submission.

Empirically, the strongest **saved offline** Stage 2 variant in the project is the embedding retriever with reranking and a soft category bonus, which reached **0.624939** in `retrieval-evaluation.ipynb`. The **current code defaults** meanwhile describe a later submission configuration with `submit_top_k=12500` and `rerank_top_m=45`. The **public Kaggle score** of **0.62337** is an external result reported by the team. The important conclusion is not that every tuned value must be copied exactly, but that the project learned the right interaction between its components: semantic retrieval provides the candidate quality, the cross-encoder repairs ranking near the top, and the category classifier helps most when used as a soft signal rather than a hard filter. That is the defensible final story of the project.